Attributes and Properties (CodeGrade)

 (https://github.com/learn-co-curriculum/python-p3-attributes-and-properties)  (https://github.com/learn-co-curriculum/python-p3-attributes-and-properties/issues/new)

Learning Goals

- Differentiate between variables, attributes, and properties.
- Use the `property()` function to create properties and validate input.

Key Vocab

- **Class**: a bundle of data and functionality. Can be copied and modified to accomplish a wide variety of programming tasks.
- **Initialize**: create a working copy of a class using its `__init__` method.
- **Instance**: one specific working copy of a class. It is created when a class's `__init__` method is called.
- **Object**: the more common name for an instance. The two can usually be used interchangeably.
- **Object-Oriented Programming**: programming that is oriented around data (made mobile and changeable in **objects**) rather than functionality. Python is an object-oriented programming language.
- **Function**: a series of steps that create, transform, and move data.
- **Method**: a function that is defined inside of a class.
- **Magic Method**: a special type of method in Python that starts and ends with double underscores. These methods are called on objects under certain conditions without needing to use their names explicitly. Also called **dunder methods** (for **d**ouble **under**score).
- **Attribute**: variables that belong to an object.
- **Property**: attributes that are controlled by methods.

Introduction

So far, we've learned how to build classes and give them instance methods. We also learned how to use the `__init__` magic method to instantiate objects and the `self` keyword to modify its **attributes**.

In this lesson, we will continue to explore attributes and **properties**, a special type of attribute.

Class Attributes vs Instance Attributes

Let's take a look at a class definition:

```
class Human:
    species = "Homo sapiens"
    def __init__(self, name):
        self.name = name
```

This class, `Human`, takes a `name` as an argument for its initialization method and saves it as an attribute of `self`. This attribute varies from one instance of the `Human` class to the next, so we call this an *instance attribute*.

Since `species` is set outside the scope of any methods, it is a *class attribute*. All members of the `Human` class have the same `species`, so this makes more sense than setting the same value for every new human upon instantation.

An interesting note about class attributes is that they can be accessed on the class itself, in addition to any instances:

```
guido = Human("Guido")
guido.species
# => Homo sapiens
Human.species
# => Homo sapiens
```

Since `name` is an instance attribute, calling it on the `Human` class will result in an error:

```
guido = Human("Guido")
guido.name
# => Guido
Human.name
# => AttributeError: type object 'Human' has no attribute 'name'
```

▶ *If we enter `guido.nationality = "Dutch"` into the interpreter, will `nationality` be a class or instance attribute?*

Setting and Getting Attributes

Many programming languages opt to protect their objects' attributes and methods (members). They accomplish this by making the distinction between *public*, *private*, and *protected*. These terms are good to know, as you will almost certainly encounter them in your software career:

- *Public* members are available to anyone that can access the class.
- *Private* members are available to the class that instantiated them.
- *Protected* members are available to the class that instantiated them and any object that inherits from that class, but is not accessible otherwise.

Python does not make the distinction between public, private, and protected. This makes it very easy for us to manipulate the members of a class or object with dot notation:

```
class Human:
    species = "Homo sapiens"
    def __init__(self, name):
        self.name = name

guido = Human("Guido")
guido.species
# => Homo sapiens
```

```
guido.name
# => Guido

# Changing species and name using dot notation
guido.species = "Python programmer"
guido.name = "Guido van Rossum"

guido.species
# => Python programmer
guido.name
# => Guido van Rossum

# Adding new attributes using dot notation
guido.nationality = "Dutch"
guido.nationality
# => Dutch
```

Because it is so simple to modify the attributes of classes and objects in Python, it is very rare that we write extra code to get or set attributes.

Python also provides us a few built-in functions to manipulate attributes:

- `getattr()` retrieves the value of an attribute.
- `setattr()` changes the value of an attribute, just as you would with dot notation.
- `hasattr()` checks for the presence of an attribute.
- `delattr()` removes an attribute from a class or object.

You might be wondering at this point why `getattr()` and `setattr()` even exist when dot notation can be used to accomplish the same tasks:

```
# Getting
guido.name
# => Guido van Rossum
```

```
getattr(guido, "name")
# => Guido van Rossum

#Setting
guido.nationality = "Dutch"
setattr(guido, "nationality", "Dutch")
```

The value in Python's `attr()` functions comes in their ability to create, retrieve, update, and delete attributes for which the names are unknown.

> NOTE: `getattr()` also allows us to provide an optional third argument as a default value if the attribute does not exist.

```
my_attr = "is_a_friend"
getattr(guido, my_attr, False)
# => False

# Oh no! Let's try again.
setattr(guido, my_attr, True)
getattr(guido, my_attr, False)
# => True
```

▶ *Which `attr()` function checks for the presence of an attribute?*

Properties

Python's flexibility with respect to members of classes and objects is very useful to us, but sometimes we need to prepare for bad actors (like me, right now):

```
# Setting Guido's age
guido.age = False
```

It is always best practice to make our code as descriptive and easy to interpret as possible. Still, there are people who may not understand what we intended or who want to break our program. It's clear that we want Guido's age to be a numerical value between 0 and some reasonable upper limit (we'll say 120). When we need to make sure an attribute meets a certain set of criteria, we need to configure it as a **property**.

Properties in Python are attributes that are controlled by methods. The function of these methods is to make sure that the value of our property makes sense. We can configure **properties** using our knowledge of object-oriented programming and Python's built-in `property()` function. Open up the Python shell or a Python file to follow along:

```
class Human:
    species = "Homo sapiens"

    def __init__(self, age):
        self.age = age

    def get_age(self):
        print("Retrieving age.")
        return self._age

    def set_age(self, age):
        print(f"Setting age to { age }")
        self._age = age

    age = property(get_age, set_age)
```

Let's break this down a bit:

- `get_age()` is compiled by the `property` function and prints `"Retrieving age"` when we access age through dot notation or an `attr()` function.
- `set_age()` is compiled by the `property()` function and prints `"Setting age to { age }"` when we change our human's age.
- The `property()` function compiles our getter and setter and calls them whenever anyone accesses our human's age.

Notice the *single underscore* we place before the age attribute. This tells other Python programmers that this is meant to be treated as a *private* member of the class. It is not truly private, but it is a way to tell your coworkers that this is a **property** and there are methods that depend on its name and values.

> NOTE: This is still not a true *private* value; you can still manipulate it with dot notation and `attr()` functions (though you shouldn't!)

There's still a problem- we're not checking if the age is a number between 0 and 120. Let's make one last change to finish our `Human` class:

```
class Human:
    species = "Homo sapiens"

    def __init__(self, age):
        self.age = age

    def get_age(self):
        print("Retrieving age.")
        return self._age

    def set_age(self, age):
        if (type(age) in (int, float)) and (0 <= age <= 120):
            print(f"Setting age to { age }.")
            self._age = age

        else:
            print("Age must be a number between 0 and 120.")

    age = property(get_age, set_age)
```

Now we have a proper **property** set up. Let's make sure it works:

```
guido = Human(age=67)
# => Setting age to 67.
guido.age = 0
# => Setting age to 0.
guido.age = False
# => Age must be a number between 0 and 120
guido.age = 66
# => Setting age to 66.
guido.age
# => Retrieving age.
# => 66
```

▶ *When should you configure a property instead of using a standard attribute?*

For more on properties, check out **the Python 3 documentation on the property() function (https://docs.python.org/3/library/functions.html#property)** .

Instructions

Fork and clone the lab and run `pytest -x` . To get the tests passing, you will need to complete the following tasks:

`Dog` and `lib/dog.py`

1. Define a `name` property for your `Dog` class. The name must be of type `str` and between 1 and 25 characters. Your `__init__` method should receive a default argument for `name` .
   - If the name is invalid, the setter method should `print()` "Name must be string between 1 and 25 characters."
2. Define a `breed` property for your `Dog` class. Your `__init__` method should receive a default argument for `breed` .

- If the breed is invalid, the setter method should `print()` "Breed must be in list of approved breeds." The breed must be in the following list of dog breeds:

```
approved_breeds = ["Mastiff", "Chihuahua", "Corgi", "Shar Pei", "Beagle", "French Bulldog", "Pug", "Pointer"]
```

`Person` and `lib/person.py`

1. Define a `name` property for your `Person` class. The name must be of type `str` and between 1 and 25 characters. The name should be converted to **title case** (https://www.w3schools.com/python/ref_string_title.asp#:%7E:text=The%20title()%20method%20returns,be%20converted%20to%20upper%20ca before it is saved. Your `__init__` method should receive a default argument for `name`.
   - If the name is invalid, the setter method should `print()` "Name must be string between 1 and 25 characters."
2. Define a `job` property for your `Person` class. Your `__init__` method should receive a default argument for `job`.
   - If the job is invalid, the setter method should `print()` "Job must be in list of approved jobs." The job must be in the following list of jobs:

```
approved_jobs = ["Admin", "Customer Service", "Human Resources", "ITC", "Production", "Legal", "Finance", "Sales", "
```

> **NOTE: Because we want to instantiate our Dogs and People with their properties, remember to include set values in `__init__()` using the *property* name and not the protected *attribute* name.**

Conclusion

Python allows us to manipulate objects very easily with dot notation and its built-in `attr()` functions. This flexibility makes it very easy to accomplish any number of tasks, but there are times when we need to be more selective about the types of changes that are saved to our objects and classes. Python's `property()` function gives us the ability to validate attributes before they are saved to the classes and objects we've worked so hard to make.

Resources

- **Class and Instance Attributes - Real Python (https://realpython.com/lessons/class-and-instance-attributes/)**
- **Property - Python Documentation (https://docs.python.org/3/library/functions.html#property)**

This tool needs to be loaded in a new browser window

Load Attributes and Properties (CodeGrade) in a new window